KamiKnows

How the pipeline works: one command per step,
every answer traceable back to the paper it came from.

E. Apicella, F. Corchia, M. Franchini



How the repository is organized

```
kamiknows/      the library: one module per step
  parsing.py  chunking.py  retrieval.py
  generation.py  review.py
  embeddings.py  common.py     (shared)
scripts/        one command per step
  parse.py  chunk.py  retrieve.py
  generate.py  review.py  chat.py
  run_ollama.slurm  index_outputs.py
tests/          offline tests of every step
data/input/     arXiv id lists (corpora)
data/questions/ questions + expected answers
docs/           outputs index (generated)
outputs/        one directory per run
```

scripts/ = how you run it

Reads the command-line options, calls one function of the library, prints the summary. Every script has `--help` with examples.

kamiknows/ = how it works

All the logic: parser, chunkers, retrievers, prompts, citation checks. The chat reuses exactly the same functions as the benchmark.

outputs/ = what each run produced

Never versioned, never overwritten. `docs/outputs_index.md` is the versioned list of all runs.

The pipeline: five steps, five commands

1. Parse

`parse.py`

Download arXiv LaTeX, extract sections and equations.

`-> sections.json`

→

2. Chunk

`chunk.py`

Split sections: word, sentence or semantic chunker.

`-> chunks.jsonl`

→

3. Retrieve

`retrieve.py`

Top-k passages per question: TF-IDF or dense.

`-> contexts.jsonl`

→

4. Generate

`generate.py`

Ollama model answers from the passages, cites them.

`-> answers.jsonl`

→

5. Review

`review.py`

Human review sheet, then label counts.

`-> review.csv`

One step, one command

Variants are options, not new scripts: `--retriever dense`, `--model qwen3:4b-instruct`.

Every run is traceable

Each run writes a new folder with `manifest.json`: command, git commit, parameters, input hashes, metrics.

Change one thing, re-run from there

A new model needs only step 4; a new chunker steps 2 to 5. Parse never needs to be repeated.

Step 1 · Parse

- Downloads metadata and the LaTeX source (e-print) of each paper.
- Inlines `\input` / `\include` and splits the text at `\section`, `\subsection`, `\subsubsection`.
- Citations become `[REF]`; display equations become `[EQUATION_001]` placeholders, saved with their LaTeX and context.
- PDF-only papers are recorded as `failed`, never guessed.

Current corpus: **5 papers, 70 sections, 28 equations** (CaloGAN, WGAN, CaloFlow, …).

COMMAND

```
python scripts/parse.py \
  --ids-file data/input/arxiv_ids_fastCalo_5fulltext.txt \
  --output-dir outputs/parsed_fastcalo
```

ID FILE

One arXiv id per line (`1705.02355v2`, `arXiv:...` or an abs URL); `#` starts a comment.

OUTPUT

```
papers.jsonl            status of every paper
papers/<id>/paper.json  metadata, warnings, errors
papers/<id>/sections.json   [{section_id, heading, text}]
papers/<id>/equations.json  [{equation_id, raw_latex, ...}]
papers/<id>/flat.tex, source/
```

Step 2 · Chunk

Chunker	How it cuts
`word`	Fixed windows of `--target-words` (400), overlap `--overlap-words` (60). A final tail under 80 words joins the previous window.
`sentence`	Packs whole sentences up to the target; the overlap is made of whole sentences.
`semantic`	Like sentence, but cuts where the embedding similarity between neighbouring sentences drops (a topic change). See next slide.

Chunks never cross sections. `--exclude-backmatter` drops acknowledgments and bibliography (logged in `section_exclusions.jsonl`).

Current datasets: **word 108**, **sentence 114**, **semantic (first drop, min 100, no back matter) 169** chunks.

COMMANDS

```
python scripts/chunk.py word \
  --parsed-dir outputs/parsed_fastcalo \
  --output-dir outputs/chunks_fastcalo_word

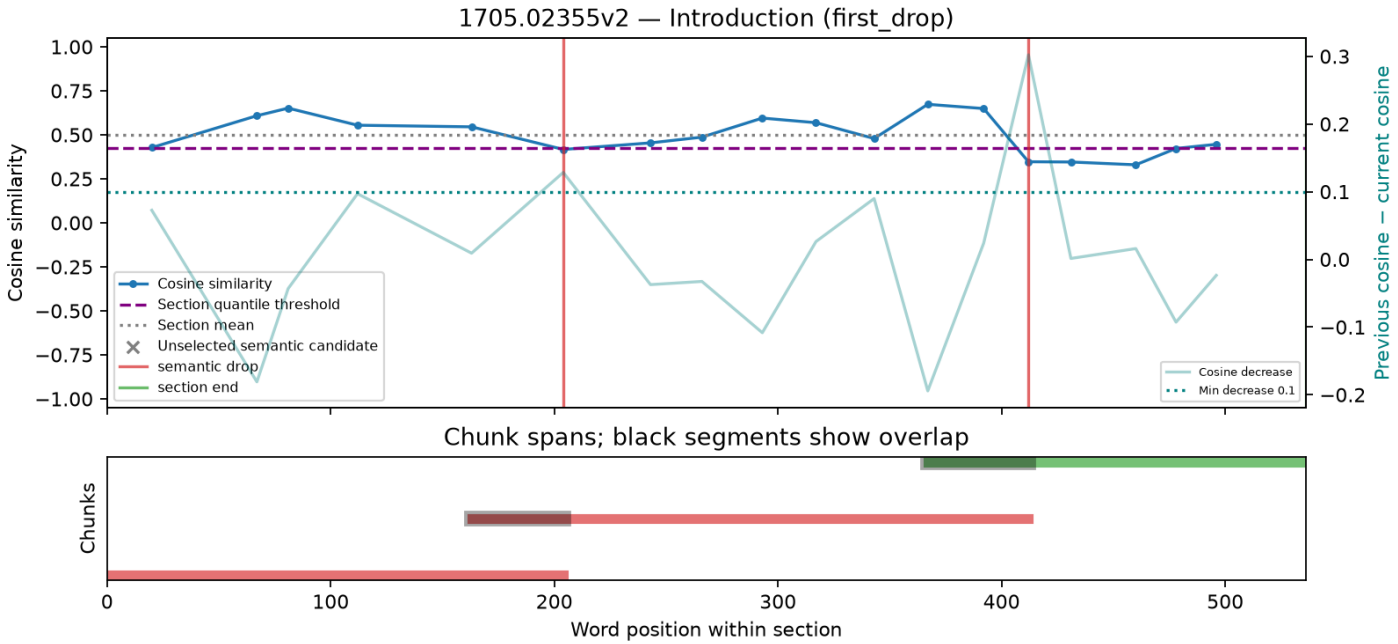
python scripts/chunk.py semantic \
  --parsed-dir outputs/parsed_fastcalo \
  --output-dir outputs/chunks_fastcalo_semantic_fd100_nobm \
  --exclude-backmatter --cut-policy first_drop \
  --min-words 100 --plots
```

EACH CHUNK IN chunks.jsonl

`chunk_id` (`<paper>::semantic::0007`), paper and section, `text`, `words`, `cut_reason` (semantic_drop, length_limit, section_end), `equation_ids` (details in `equations.jsonl`), `flags`.

Semantic chunking: where to cut

- Each sentence becomes a vector (`all-MiniLM-L6-v2`).
- At every sentence boundary: cosine similarity between the 2 sentences before and the 2 after.
- **Candidate** cut: similarity in the lowest 25% of its section.
- `first_drop`: cut at the first candidate where the similarity fell by at least 0.10 from the previous boundary.
- The chunk must stay within `--min-words` and `--target-words`; otherwise it ends at the size limit or the section end.



With `--plots`: one figure per section plus an overview comparing word / sentence / semantic chunk lengths.

Step 3 · Retrieve

TF-IDF

Weighted word counts, compared word by word. Title and section heading count twice.

Dense

Cosine between MiniLM embeddings of the question and of "title section text". Model: `--embedding-model`.

Source policy

`explicit_arxiv` (default): if the question names an arXiv id, only that paper is searched and the best chunk of each named paper is always kept. A paper missing from the corpus gives no passages, so the model must abstain.

The query is **the question text only**. Expected papers and behaviour in the questions file are used to score retrieval, never to rank.

COMMAND

```
python scripts/retrieve.py \
  --dataset-dir outputs/chunks_... \
  --questions data/questions/\
      fastcalo_paired.jsonl \
  --output-dir outputs/retrieval_... \
  --retriever dense --top-k 5
```

CURRENT RESULTS (16 questions)

Retriever	hit@5	MRR
TF-IDF	0.94	0.72
Dense	0.88	0.66

`hit@5`: an expected paper is among the 5 passages. MRR: 1/rank of the first hit, averaged.

Step 4 · Generate

COMMAND (on a GPU node, see SLURM)

```
python scripts/generate.py \
  --retrieval-dir outputs/retrieval_... \
  --output-dir outputs/qa_... \
  --model qwen3:4b-instruct \
  --prompt grounded          # or synthesis
```

- The model sees exactly the saved passages `[C1]...[C5]`: retrieval is not re-run.
- `grounded`: answer only if supported, otherwise abstain. `synthesis`: may combine passages and give marked partial answers.
- `--question-ids` re-asks only some questions (e.g. with another prompt).
- Answers are saved one by one: an interrupted job keeps what it finished.

EACH ANSWER


```
{"answer_status": "answered",
 "answer": "CaloGAN, a GAN ... [C1]",
 "citations": ["C1", "C3"], "confidence": "high",
 "citation_validation": {"status": "valid"},
 "warnings": [], "generation": {"eval_count": 312}}
```


Citation check: `valid`, `invented_citation` (label not among the passages), `answered_without_citation`. It checks labels, not whether the passage supports the claim.

CURRENT RESULT (qwen3:4b-instruct, dense, 20 questions)

	count
Answered / abstained	11 / 9
Valid citations	20 / 20
Correct refusals (no answer in corpus)	4 / 4
Abstained on answerable questions	5

Step 5 · Review

- One CSV for one or more generation runs: side-by-side comparison of models, prompts, retrievers.
- Each row: question, expected behaviour, **reference answer**, required points and evidence (from the questions file), the model answer, the citation check, the text of the cited passages.
- Reviewers fill three labels and notes, then set `review_status`.

Column	Allowed values
`reviewer_correctness`	correct, partially_correct, incorrect, not_applicable
`reviewer_citation_support`	supported, partially_supported, unsupported, not_applicable
`reviewer_abstention`	appropriate, should_have_answered, should_have_abstained, not_applicable

COMMANDS

```
python scripts/review.py prepare \
  --qa-dirs outputs/qa_A outputs/qa_B \
  --output-dir outputs/review_A_vs_B

# after filling review.csv
python scripts/review.py summarize \
  --csv outputs/review_A_vs_B/review.csv
```

Why it matters. Automatic checks validate formats and citation labels. Only the review says whether an answer is scientifically correct.

Running on SLURM, and the chat

GENERATION ON A GPU NODE

```
sbatch --job-name=KK-qa-qwen4b scripts/run_ollama.slurm \
  scripts/generate.py \
  --retrieval-dir outputs/retrieval_... \
  --output-dir outputs/qa_... --model qwen3:4b-instruct
```

- Starts the shared Ollama from `/storage-hpc/kamiknows` on a port derived from the job id.
- Waits until it answers, runs the command and appends `--base-url`.
- Copies `slurm.log` and `ollama.log` into the run folder. Live log: `outputs/_slurm_logs/`.
- 1 GPU, 4 CPUs, 16 GB, 1 h by default; override with `sbatch --time=02:00:00`.

Parse, chunk and retrieve run on the login node (about a minute on CPU).

INTERACTIVE CHAT

```
srun --account=kamiknows \
  --qos=kamiknows --gres=gpu:1 \
  --partition=bare-metal-GPU --pty bash
# start the shared Ollama server, then
python scripts/chat.py \
  --dataset-dir outputs/chunks_... \
  --model qwen3:4b-instruct
```

Same retrieval and generation as the benchmark. Each question is independent; every turn is saved in `outputs/chat/`.

Every run is traceable

manifest.json (in every run folder)

```
{"step": "retrieve",
 "created_at": "2026-10-21T14:19:12Z",
 "command": ["python", "scripts/retrieve.py", ...],
 "git": {"commit": "46d6999...", "dirty": false},
 "params": {"retriever": "dense", "top_k": 5, ...},
 "inputs": {"chunks": {"path": ..., "sha256": ...},
            "questions": {...}},
 "summary": {"hit_rate": 0.875, "mrr": 0.66, ...}}
```

- `--output-dir` must be new: scripts refuse to overwrite; retry as `..._retry1`.
- Each step checks that its input folder comes from the previous step.

NAMING PATTERN

```
parsed_<corpus>
chunks_<corpus>_<chunker>
retrieval_<chunker>_<retriever>_<sources>_<evalset>_top<k>
qa_<chunker>_<retriever>_<sources>_<model>_<evalset>_top<k>
review_<what is reviewed>
```

Tag	Meaning
`fd100`	cut policy first_drop, at least 100 words per chunk
`nobm`	no back matter (acknowledgments, bibliography)
`dense`	embedding retriever (MiniLM); `tfidf` lexical
`explicit`	source policy explicit_arxiv; `allsrc` = none
`paired`	questions file data/questions/fastcalo_paired.jsonl
`qwen4b`	model qwen3:4b-instruct

Questions files and outputs index

- One evaluation = **one JSONL file** in `data/questions/`: questions and expected answers together.
- **Never edit a file that runs have used**: copy it to a new name (e.g. `fastcalo_paired_hard.jsonl`), edit the copy, use a new tag in run names.
- Expected fields score and support the review; they never reach the ranking or the prompt.

Field	Used for
`question_id`, `question`	required
`expected_behavior`	answerable / insufficient_evidence
`expected_arxiv_ids`	retrieval `hit@k` and MRR
`reference_answer`, `required_answer_points`	review sheet

CURRENT SET: fastcalo_paired.jsonl

10 pairs: each question once naming its arXiv id and once without. 16 answerable, 4 that must be refused. Reference answers are drafts.

OUTPUTS INDEX

`outputs/` is not in git; `docs/outputs_index.md` is: one table per step, built from the manifests. Notes per run go in `docs/outputs_index_notes.json`.

```
# after every run
python scripts/index_outputs.py
```

Quick start

```
conda activate kamiknows        # pip install -r requirements.txt
P=outputs/parsed_fastcalo  C=outputs/chunks_fastcalo_semantic_fd100_nobm
R=outputs/retrieval_semantic_fd100_nobm_dense_explicit_paired_top5
Q=outputs/qa_semantic_fd100_nobm_dense_explicit_qwen4b_paired_top5

python scripts/parse.py    --ids-file data/input/arxiv_ids_fastCalo_5fulltext.txt --output-dir $P
python scripts/chunk.py semantic --parsed-dir $P --output-dir $C \
       --exclude-backmatter --cut-policy first_drop --min-words 100 --plots
python scripts/retrieve.py --dataset-dir $C --questions data/questions/fastcalo_paired.jsonl \
       --output-dir $R --retriever dense --top-k 5
sbatch scripts/run_ollama.slurm scripts/generate.py --retrieval-dir $R --output-dir $Q \
       --model qwen3:4b-instruct
python scripts/review.py prepare --qa-dirs $Q --output-dir outputs/review_qwen4b_paired
python scripts/index_outputs.py

pytest -q                       # offline tests of every step
```

Full reference: `README.md`. Rules for coding agents: `AGENTS.md`. Every script: `python scripts/<step>.py --help`.